Assignment

API Data Collection and Time-Series Visualization

Learning Objectives

• Demonstrate proficiency in making HTTP GET requests to RESTful APIs

• Extract and parse JSON data structures from API responses

• Handle API authentication and error management

• Process time-series data with proper datetime handling

• Create professional visualizations of energy/environmental data

• Document code with clear comments and explanations

Assignment Overview

In this assignment, you will demonstrate your ability to work with real-world APIs by extracting time-series data and creating professional visualizations. You must choose ONE of the APIs provided in the lab tutorial, fetch relevant time-series data, and create at least THREE different time-series plots.

Requirements Summary

This assignment requires you to submit a single Jupyter notebook (.ipynb file) containing:

• API selection and justification

• Working code with error handling

• Three time-series visualizations (detailed below)

• Analysis and interpretation for each plot

• Clean, documented code with reusable functions

Deliverable: Submit your completed Jupyter notebook (.ipynb file) with all code, plots, and analysis.

API Options

Choose ONE of the following APIs covered in the lab tutorial:

API	Data Available	Time Resolution
REE (Spain) Red Eléctrica de España	• Electricity spot prices • PVPC regulated prices • Generation mix • Demand data	Hourly, Daily
CO2 Signal / Electricity Maps	• Carbon intensity • Fossil fuel percentage • Power breakdown • Import/export data	Hourly, Historical
Weather API (RapidAPI)	• Temperature • Wind speed • Humidity • Precipitation	Current, Historical

Detailed Requirements

1. Time-Series Plot #1: Basic Line Plot (20 points)

Description: Create a basic time-series line plot showing the evolution of a single variable over time.

- **Data requirements:** Minimum 24 hours of data, ideally 1 week or more
- **Must include:** Title, axis labels with units, grid, legend (if applicable)
- **Additional elements:** Average line or trend indicator
- **DateTime handling:** Proper datetime objects on x-axis with readable formatting

Examples:

- Electricity spot prices over 1 week
- Carbon intensity for a country over 48 hours
- Temperature evolution over 1 month

2. Time-Series Plot #2: Comparative Plot (20 points)

Description: Create a plot comparing multiple time-series on the same chart or using subplots.

- **Data requirements:** At least 2 different variables or same variable for different entities
- **Must include:** Clear differentiation (colors, line styles, markers)
- **Additional elements:** Legend explaining each series
- **Analysis:** Brief text interpretation of the comparison

Examples:

- PVPC vs Spot market prices
- Carbon intensity comparison between 3 countries
- Temperature vs Wind speed correlation
- Weekend vs Weekday electricity prices

3. Time-Series Plot #3: Advanced Visualization (20 points)

Description: Create an advanced visualization showing aggregated data, breakdowns, or derived insights.

- **Data requirements:** Aggregated or processed time-series data
- **Visualization types:** Stacked area, heatmap, box plot by period, or dual-axis
- **Must include:** Professional formatting and clear data story
- **Insights:** Statistical analysis or key findings displayed on plot

Examples:

- Power generation breakdown (stacked area chart)
- Hourly price patterns aggregated by day of week (heatmap)
- Monthly average comparison with variance bands
- Daily min/max/average statistics (box plot)

Technical Requirements

Error Handling: All API calls must include try-except blocks with meaningful error messages

Status Codes: Check and handle HTTP status codes appropriately

Datetime Handling: Use Python datetime objects, not strings, for time-series data

Functions: Create reusable functions for data fetching and plotting

Comments: Add docstrings and inline comments explaining your logic

Code Structure: Organize code logically with clear sections

Code Quality Guidelines (10 points)

• Use meaningful variable names (e.g., `electricity_prices` not `data`)

• Create helper functions to avoid code repetition

• Include docstrings for all functions explaining parameters and returns

• Add comments explaining complex logic or API-specific quirks

• Follow PEP 8 style guidelines (proper spacing, line length, etc.)

• Handle edge cases (empty responses, missing data, API errors)

Grading Rubric

Category	Excellent (90-100%)	Good (75-89%)	Needs Improvement (<75
API Selection (10 pts)	Well-justified choice appropriate for analysis	API selected but limited justification	Poor fit or no justification
Data Extraction (15 pts)	Robust error handling, clean data fetching	Basic extraction works, minor issues	Incomplete or failing code
Plot Quality (60 pts total)	Professional plots with all required elements	Plots functional but lacking polish	Missing elements or poor visualization
Code Quality (10 pts)	Clean, documented, reusable functions	Functional but could be more organized	Poorly structured, lack of comments
Documentation (5 pts)	Complete README with clear explanations	README present but missing details	Minimal or no documentation

Tips for Success

• **Start Early:** API issues can arise - give yourself time to troubleshoot

• **Test Incrementally:** Don't write all code at once - test each step

• **Read Documentation:** Each API has quirks - understand them first

• **Handle Missing Data:** APIs may return null/None values - handle gracefully

• **Save Your Work:** Cache API responses during development to avoid rate limits

• **Make It Visual:** Use colors, markers, and styling to make plots engaging

• **Tell a Story:** Your plots should communicate insights, not just display data

• **Ask Questions:** If stuck, consult documentation or ask for clarification

Common Pitfalls to Avoid

■ **Not checking status codes** - Always verify successful responses

■ **Ignoring time zones** - Be aware of UTC vs local time

■ **Plotting strings as dates** - Convert to datetime objects first

■ **No error handling** - API calls can fail for many reasons

■ **Hardcoded API keys** - Use variables or environment variables

■ **Overcomplicating plots** - Sometimes simpler is better

Helpful Resources

Resource	URL	Purpose
Matplotlib Gallery	matplotlib.org/stable/gallery	Plot inspiration
JSON Formatter	jsonformatter.org	Visualize API responses
Datetime strftime	strftime.org	Format datetime strings
Requests Docs	docs.python-requests.org	HTTP library reference
Lab Tutorial	Provided Jupyter notebook	Code examples

Good luck with your assignment! ■